Aquisição de dados do DYNSIM e do PI Server com Python

Procedimentos de configuração e leitura em Windows · Revisão de 14 de outubro de 2026

A aquisição é realizada por duas rotas: leitura das variáveis atuais de uma simulação DYNSIM por OPC DA e extração dos eventos armazenados no PI Data Archive pelo AF SDK. O documento organiza os requisitos, a configuração das conexões, a execução dos programas e a validação dos resultados.

1 Escopo e fluxo de dados

Na rota OPC DA, o motor de processo Dynsim transfere a variável medida de FC101 ao servidor Matrikon por meio do motor OPC01. O programa Python consulta esse servidor. Na rota PI, o notebook acessa pontos já cadastrados e historizados no Data Archive e produz um CSV por tag.

Aspecto	DYNSIM por OPC DA	PI Server pelo AF SDK
Origem	Variáveis atuais da simulação	Eventos existentes no histórico PI
Integração Python	OpenOPC e OPC Automation por COM	pythonnet e OSIsoft.AFSDK.dll
Ambiente adotado	Python 3.12 de 32 bits	Python 3.12 de 64 bits
Consulta	Pontual ou contínua	Intervalo definido de início e fim
Saída do código	Valor, qualidade e horário OPC na console	CSV com Tag, Timestamp e Value

Um tag identifica uma variável. No OPC DA, seu identificador completo é o Item ID; no PI, a variável corresponde a um PI Point. A igualdade entre nomes nas duas plataformas deve ser conferida. A criação de um alias em Matrikon, isoladamente, não cadastra nem historiza um ponto no PI.

Organização do procedimento

As seções 2 a 8 descrevem a leitura direta do DYNSIM. As seções 9 a 11 descrevem a extração do PI e podem ser executadas independentemente da configuração OPC. As seções 12 e 13 estabelecem critérios de validação e diagnóstico comuns às duas rotas.

O escopo pressupõe um caso DYNSIM existente, Matrikon instalado e acesso a um PI Data Archive operacional. A instalação do simulador, a implantação do PI Server e a configuração da interface que historiza dados no PI devem estar concluídas antes da aquisição.

2 Requisitos e organização do ambiente

Os programas de aquisição são executados em Windows. Para o OPC DA local, DYNSIM, Matrikon e o leitor Python permanecem na mesma estação. Se o simulador estiver em uma sessão de área de trabalho remota, o leitor é executado dentro dessa estação. localhost designa o computador onde o cliente está em execução.

Componente	Configuração de referência
Simulador e caso	AVEVA Dynamic Simulation 2023 e DEETHA_HB02
Motores DYNSIM	Dynsim para o processo e OPC01 do tipo OPCClient
Servidor OPC	Matrikon.OPC.Simulation.1
Cliente de conferência	Matrikon OPC Explorer
Cliente PI	PI AF Client com AF SDK e acesso autorizado ao Data Archive
Arquivos de execução	leer_matrikon_opc_da.py e Extrator_Dados_PI.ipynb

1. Extrai-se o pacote de arquivos em uma pasta de trabalho, por exemplo Coleta_Dados dentro de Documentos. Os dois arquivos de execução devem permanecer nessa pasta. O README do pacote registra seus nomes e sua função.

2. No Explorador de Arquivos, abre-se a pasta de trabalho, digita-se powershell na barra de endereços e pressiona-se Enter. Os comandos seguintes são executados nessa console; as linhas que começam com PS na saída do Windows não fazem parte dos comandos.

3. Instala-se Python 3.12.10 nas arquiteturas de 32 e 64 bits pela página oficial [4], mantendo o Python Launcher disponível. Essa versão reproduz o ambiente OPC confirmado. Para PI, o notebook original registra Python 3.12.7; a adoção de 3.12.10 deve ser validada pela importação do AF SDK.

`https://www.python.org/downloads/release/python-31210/`

Em Files, selecionam-se Windows installer (32-bit) e Windows installer (64-bit), conforme o ambiente. Se os intérpretes já estiverem instalados, realiza-se apenas a conferência:

```
py -0p
py -3.12-32 -c "import struct; print(struct.calcsize('P') * 8)"
py -3.12-64 -c "import struct; print(struct.calcsize('P') * 8)"
```

Verificação: os dois últimos comandos retornam 32 e 64, respectivamente. As dependências de cada rota serão instaladas em ambientes virtuais próprios nas seções 8 e 9. A presença de uma pasta Program Files (x86) no caminho do AF SDK não determina, isoladamente, a arquitetura do interpretador.

3 Criação dos aliases no Matrikon

O caso utiliza FC101.PV, a vazão medida, e FC101.SP, a referência do controlador. Na interface do controlador, ambas aparecem em m^3/hr. No servidor, esses valores são representados por aliases que aceitam leitura e escrita.

1. Abre-se MatrikonOPC Server for Simulation and Testing e seleciona-se Alias Configuration no painel esquerdo.

2. Em Edit > Insert Alias Group, cria-se Process. Com esse grupo selecionado, cria-se Plant01. Dentro dele, cria-se Control e, em seguida, FC101. Confirma-se cada nome com Enter.

3. Com FC101 selecionado, utiliza-se Edit > Insert New Alias para cadastrar PV e SP. Os campos são preenchidos conforme a tabela e cada alias é confirmado com Save.

Campo	Alias PV	Alias SP
Name	PV	SP
Item Path	Vazio	Vazio
Data Type	REAL8	REAL8
Read only	Desmarcado	Desmarcado
Scaling	Sem escalonamento	Sem escalonamento

REAL8 representa um valor real de dupla precisão. Com Item Path vazio e tipo explícito, o alias mantém o valor em memória. O acesso R/W permite que DYNSIM publique a PV no servidor [1].

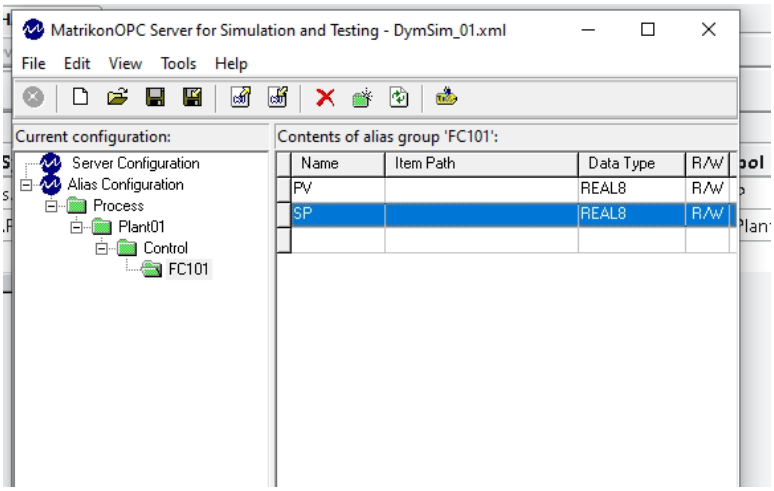


Figura 1. Hierarquia de aliases utilizada no caso FC101.

4. Em File > Save As, salva-se a configuração do servidor como DymSim_01.xml. Registra-se sua localização para que o arquivo possa ser carregado nas sessões seguintes.

Verificação: PV e SP aparecem dentro de Process > Plant01 > Control > FC101 com tipo REAL8 e acesso R/W. O XML contém a configuração de Matrikon; o arquivo de comunicação de DYNSIM será gerado separadamente na seção 5.

4 Validação dos tags no OPC Explorer

1. Abre-se Matrikon OPC Explorer. No painel esquerdo, localiza-se Matrikon.OPC.Simulation.1 e seleciona-se Connect no menu de contexto. Confirma-se Connected = Yes.

2. Utiliza-se Add Tags para abrir o navegador de tags. Se um grupo já existir, seleciona-se o grupo e utiliza-se Group > Add Items ou o botão Add OPC Items. Mantém-se o nome de grupo atribuído pelo servidor, como Group0.

3. No campo Item ID, insere-se o primeiro identificador abaixo e utiliza-se a seta de inclusão em Tags to be added. Repete-se a operação com o segundo identificador.

```
Process.Plant01.Control.FC101.PV
Process.Plant01.Control.FC101.SP
```

4. Executa-se File > Validate Tags ou Validate Items. Após a validação, seleciona-se File > Update and Return para retornar à visualização dos valores. Uma marca verde indica um item validado [2].

5. Antes de habilitar o XRef, seleciona-se SP e utiliza-se Write Values no menu de contexto. Para o caso de referência, insere-se 85 em New Value e confirma-se com OK. Essa referência será transmitida ao controlador quando a relação de entrada estiver ativa.

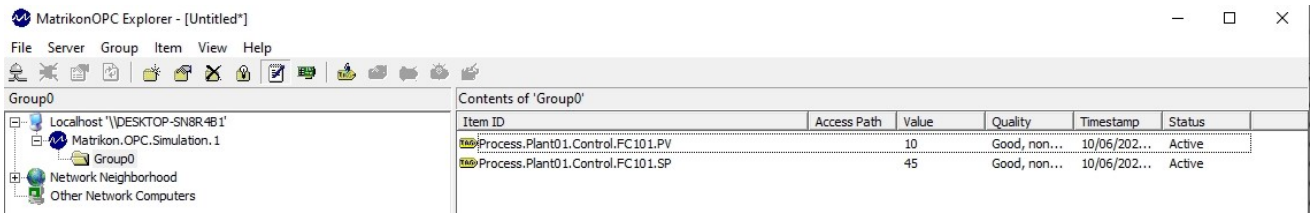


Figura 2. Item ID dos dois tags em Explorer. Os valores exibidos pertencem a uma configuração inicial.

Verificação: os dois tags apresentam estado ativo e qualidade Good; após a escrita, SP apresenta 85. A PV passa a acompanhar o processo após a configuração e execução dos motores. A variação da PV será verificada na seção 7.

O valor 85 pertence ao ponto de operação do exemplo. Em outro caso, a referência é definida de acordo com as condições da simulação. Os Item ID completos devem coincidir com aqueles utilizados no configurador e no programa Python.

5 Geração do arquivo OCF

OPC01.ocf armazena a conexão e os itens do motor OPCClient. O arquivo é produzido pelo OPC Configurator de DYNSIM. Sua geração exige que Matrikon esteja disponível com os aliases definidos na seção 3.

1. Abre-se OPC Configurator e cria-se uma configuração pelo menu File ou pelo ícone de folha em branco.

2. Na configuração de conexão, selecionam-se o computador local e Matrikon.OPC.Simulation.1. Estabelece-se a conexão com o servidor.

3. Em Browse View, na região inferior esquerda, localizam-se os dois Item ID. Pela ação de inclusão de itens, associa-se PV a Model Inputs e SP a Model Outputs.

Tag	Lista no OPCClient	Transferência
PV	Model Inputs	Valor do motor Dynsim para o servidor Matrikon
SP	Model Outputs	Valor do servidor Matrikon para o motor Dynsim

As capturas disponíveis registram a configuração concluída, mas não mostram os diálogos de seleção do servidor e de inclusão de itens. Os nomes exatos dessas duas opções permanecem pendentes de confirmação na instalação utilizada. Essa limitação afeta os passos 2 e 3.

4. Em OPC View, expande-se OPC Group Configuration e verifica-se o conteúdo das duas listas. State Variables permanece sem esses itens.

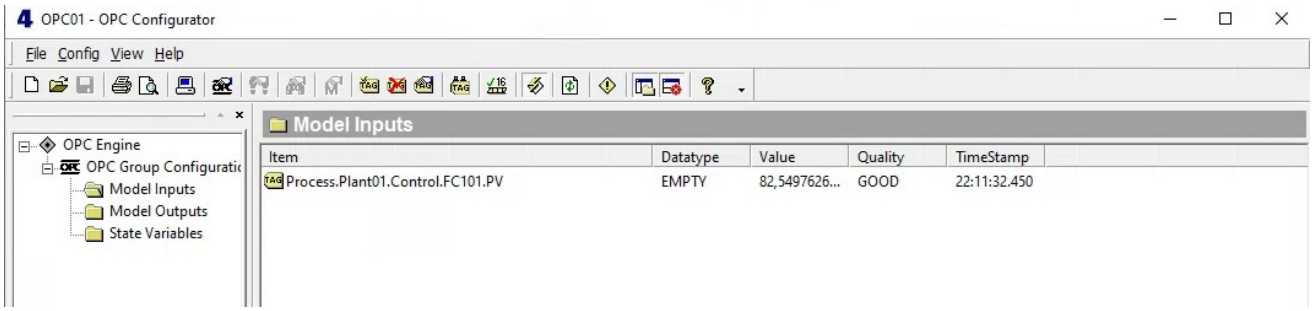


Figura 3. PV cadastrada em Model Inputs.

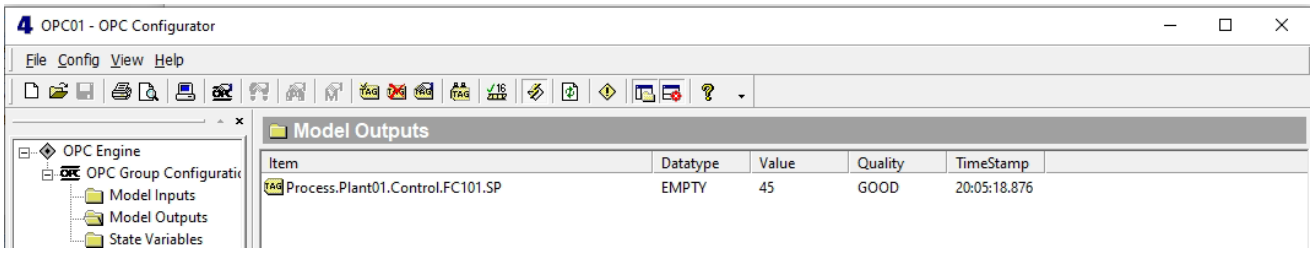


Figura 4. SP cadastrada em Model Outputs. O valor 45 corresponde à captura inicial.

5. Pelo menu File, salva-se a configuração como OPC01.ocf no diretório abaixo. Cria-se a pasta OPC01 se ela ainda não existir.

```
C:\SimSci\DSS64\Simulations\DEETHA_HB02\OPC01\OPC01.ocf
```

Verificação: o OCF existe nessa localização e conserva PV e SP ao ser reaberto no configurador. Os nomes do caso e do diretório são ajustados se outra simulação for utilizada.

6 Configuração do motor OPCClient

1. Se os motores estiverem iniciados, executa-se Shut Down antes da edição. Em Simulation Configuration, preserva-se o motor Dynsim e adiciona-se um motor de comunicação com Add. Se OPC01 já existir, seleciona-se Edit.

2. Os campos do motor são preenchidos conforme a tabela. Engine Host recebe o nome do computador Windows em que o motor será executado.

Campo	Valor do caso
Engine Name	OPC01
Engine Type	OPCClient
Engine Host	Nome da estação Windows
OPC file path	Caminho completo de OPC01.ocf da seção 5
Engine time step	0.25
Reload OPC file on LI and LF	1
localhost or IP address	localhost

O nome da estação pode ser obtido no PowerShell com:

```
$env:COMPUTERNAME
```

3. Aplicam-se os parâmetros, habilita-se Load engine para Dynsim e OPC01 e salva-se a configuração da simulação. O motor é iniciado novamente para aplicar alterações realizadas após Shut Down.

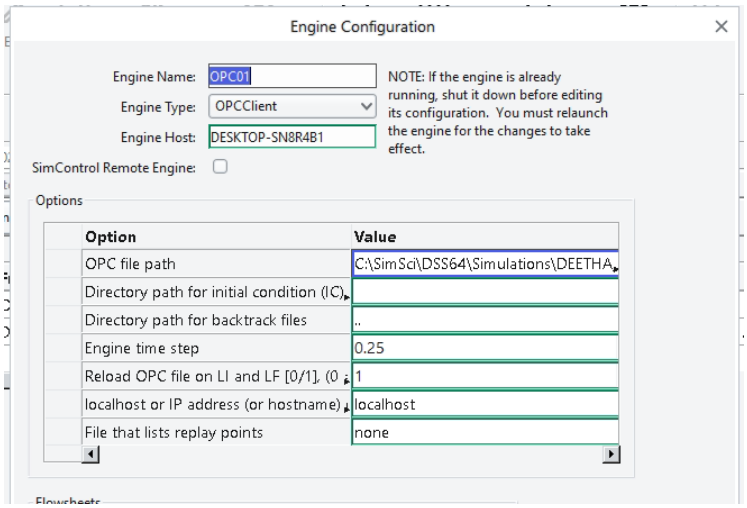


Figura 5. Configuração do motor OPC01.

Verificação: Simulation Configuration contém os dois motores e OPC file path aponta para o arquivo existente. OPC01 será utilizado exatamente com esse nome no XRef. Os parâmetros apresentados reproduzem o caso documentado.

7 Transferência das variáveis pelo XRef

O Cross Reference, ou XRef, define o sentido da transferência entre motores. From identifica a origem; To identifica o destino. A PV é publicada pelo processo, enquanto o SP é recebido de Matrikon.

1. Abre-se Cross Reference e, na aba Analog, utiliza-se Insert para cadastrar as duas relações da tabela.

From Engine	From Symbol	To Engine	To Symbol
OPC01	Process.Plant01.Control.FC101.SP	Dynsim	FC101.SP
Dynsim	FC101.PV	OPC01	Process.Plant01.Control.FC101.PV

2. Disabled permanece desmarcado nas duas relações. Salva-se a configuração e encerra-se a edição. O nome OPC101, identificado no primeiro ensaio, deve ser corrigido para OPC01 caso apareça em alguma relação.

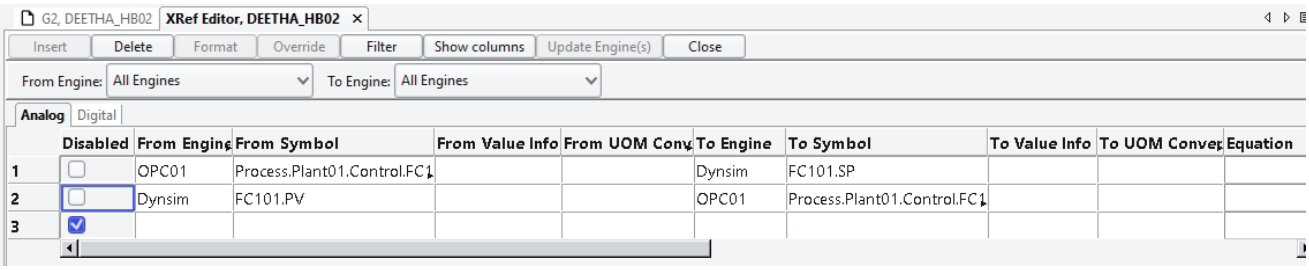


Figura 6. Relações analógicas habilitadas no XRef.

3. Executa-se Load Full para carregar o caso e seus motores; em seguida, executa-se Load XRef. Em Messages, verifica-se a ausência de erros relacionados ao motor e aos símbolos.

4. No controlador FC101, seleciona-se AUTO para a operação automática e executa-se Run. Confirma-se RUNNING e o avanço do tempo de simulação.

5. Compara-se a PV do controlador com a PV de Explorer durante uma variação do processo. O SP deve coincidir com a referência definida no servidor. Não se escreve manualmente a PV durante essa conferência.

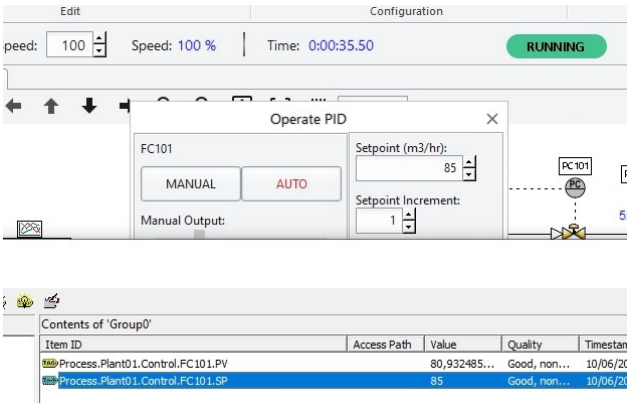


Figura 7. Execução do caso e leitura das variáveis em Explorer.

Verificação: a PV acompanha o controlador e apresenta qualidade Good em Matrikon. Qualidade válida, isoladamente, não comprova a chegada de novos valores da simulação; essa conclusão depende da comparação durante a variação da PV.

8 Leitura direta do DYNSIM em Python

Preparação e execução

1. No PowerShell aberto na pasta do pacote, cria-se o ambiente OPC de 32 bits e instalam-se suas dependências. O arquivo leer_matrikon_opc_da.py permanece nessa pasta.

```
py -3.12-32 -m venv .venv_opc
.\.venv_opc\Scripts\python.exe -m pip install OpenOPC-DA==1.5.1 pywin32 Pyro5
```

O acesso direto utiliza OpenOPC e pywin32 com um componente OPC Automation registrado no Windows [3]. No ensaio original, a criação do cliente COM falhou em Python de 64 bits e funcionou em Python 3.12.10 de 32 bits. O registro da DLL não foi inspecionado nesse ensaio.

2. Executa-se uma leitura pontual para validar o cliente Python:

```
.\.venv_opc\Scripts\python.exe leer_matrikon_opc_da.py --once
```

Verificação: a conexão é estabelecida com Matrikon.OPC.Simulation.1 em localhost e são exibidos PV e SP com qualidade Good. O resultado confirmado apresentou PV = 82.402079 e SP = 85.000000; os valores variam conforme o estado do caso.

3. Para a leitura contínua, executa-se o comando abaixo. A execução termina com Ctrl+C ou com uma exceção de comunicação.

```
.\.venv_opc\Scripts\python.exe leer_matrikon_opc_da.py --interval 1
```

--once encerra o programa após uma leitura. --interval 1 define uma espera de um segundo entre consultas; a duração da comunicação é adicional. Para registrar a saída textual no PowerShell, utiliza-se:

```
.\.venv_opc\Scripts\python.exe leer_matrikon_opc_da.py --interval 1 |
    Tee-Object -FilePath .\opc_da.log
```

O arquivo opc_da.log contém texto de console e não constitui um CSV. O leitor fornecido não grava uma tabela estruturada nem recupera um histórico de eventos; ele consulta os valores atuais dos tags.

Arquitetura do leitor

As entradas são o ProgID do servidor, o host, a lista TAGS e o intervalo. O módulo de conexão cria o cliente COM; o módulo de leitura cria um grupo e o reutiliza em consultas síncronas com source="device". O módulo de apresentação exibe valor, qualidade e horário OPC. O encerramento remove o grupo e fecha o cliente.

A ampliação da coleta exige cadastrar e vincular os novos aliases nas seções 3, 5 e 7 e inserir seus Item ID em TAGS. O programa apenas lê os tags; o SP é escrito por Explorer na configuração deste caso.

9 Preparação da extração do PI Server

A rota PI utiliza o notebook Extrator_Dados_PI.ipynb, desenvolvido por José Matheus Mendes Trindade. O programa consulta o PI Data Archive por meio das bibliotecas do AF SDK. A estação deve possuir PI AF Client, acesso à rede interna ou VPN e permissões de leitura para os pontos solicitados.

1. Confirma-se a instalação do AF SDK e a existência de OSIsoft.AFSDK.dll. O caminho padrão utilizado pelo notebook é:

```
C:\Program Files (x86)\PIPC\AF\PublicAssemblies\4.0
```

A instalação do cliente PI é obtida pelo canal institucional ou pelo administrador do sistema. As dependências instaladas por pip fornecem a integração Python; elas não instalam o AF SDK.

2. Na pasta de trabalho, cria-se o ambiente de 64 bits e instalam-se os pacotes do notebook:

```
py -3.12-64 -m venv .venv_pi
.\.venv_pi\Scripts\python.exe -m pip install pandas pythonnet jupyter ipykernel
```

3. Registra-se o kernel com o identificador interno pi_afsdk e o nome visível PI AF SDK 64 bits. Executa-se o bloco completo; o acento grave no final da primeira linha indica continuação no PowerShell [8]:

```
.\.venv_pi\Scripts\python.exe -m ipykernel install --user --name pi_afsdk `
    --display-name "PI AF SDK 64 bits"
```

O kernel é o processo Python que executa as células. Seu ambiente deve conter os pacotes instalados no passo 2. No Windows, pythonnet utiliza .NET Framework por padrão; a carga da biblioteca deve ser compatível com o AF SDK instalado [5].

4. Confere-se o Data Archive configurado no cliente PI. O notebook usa servidores.DefaultPIServer. Na saída registrada, esse servidor era 164.41.16.87; em outra instalação, utiliza-se o nome ou endereço autorizado pelo administrador.

O código não fornece usuário e senha. O acesso depende da configuração de autenticação do cliente e das permissões do usuário Windows. A resposta a ping indica apenas resposta ICMP; a leitura de um ponto pela biblioteca é a verificação da comunicação de aplicação.

Verificação: a DLL está instalada, o ambiente retorna 64 bits e o servidor PI esperado está configurado no cliente. A carga do SDK e a leitura efetiva serão conferidas na execução das células.

10 Configuração do notebook PI

O notebook contém três células de código com dependências sequenciais. As células de texto descrevem o programa e não substituem sua execução. Os blocos possuem as seguintes responsabilidades:

Bloco	Entradas e processamento	Saída
Célula 1	Importa pandas e clr, acrescenta caminho_sdk e carrega OSIsoft.AFSDK	Classes PIServers, PIPoint, AFTime, AFTimeRange e AFBoundaryType
Célula 2	Define extrair_e_salvar_tag_csv e consulta eventos por RecordedValues	Um DataFrame e um CSV por tag
Célula 3	Seleciona o servidor, o período e tags_para_buscar; percorre a lista	Execução das extrações e mensagens por tag

1. Na Célula 1, ajusta-se caminho_sdk apenas se a DLL estiver em outro diretório. O caminho precisa apontar para a pasta que contém a biblioteca.

2. Na Célula 3, confere-se meu_servidor. Caso o servidor padrão não seja o destino correto, substitui-se a atribuição por um servidor já cadastrado no cliente. Exemplo correspondente ao ambiente documentado:

```
servidores = PIServers()
meu_servidor = servidores["164.41.16.87"]
```

3. Define-se um período que possua dados no arquivo histórico. Datas absolutas facilitam a repetição da consulta. O bloco abaixo apresenta um exemplo de configuração, e não uma garantia de disponibilidade nesse período:

```
data_inicio = "2026-03-01 00:00:00"
data_fim = "2026-10-06 23:59:59"
tags_para_buscar = [
    "Process.Plant01.Control.FC101.PV",
    "Process.Plant01.Control.LC101.PV",
]
```

O caractere * pode ser utilizado em data_fim para consultar até o instante atual. Como o notebook converte esse parâmetro dentro da função de cada tag, as extrações podem obter instantes finais ligeiramente diferentes. Um horário final absoluto mantém o mesmo limite entre variáveis.

4. Confere-se a lista de pontos no PI. O exemplo histórico inclui LC101.PV, enquanto o leitor OPC consulta FC101.SP. A seleção de variáveis deve corresponder ao objetivo da aquisição, sem presumir que todos os aliases OPC tenham um ponto PI equivalente.

Verificação: o diretório do SDK está correto, o servidor corresponde ao destino autorizado, o instante inicial precede o final e cada tag existe no Data Archive.

11 Execução e resultados da extração PI

Execução pelo Jupyter

1. Na pasta que contém Extrator_Dados_PI.ipynb, inicia-se Jupyter com o interpretador do ambiente PI:

```
.\.venv_pi\Scripts\python.exe -m jupyter notebook
```

2. Na interface do navegador, abre-se o notebook e seleciona-se PI AF SDK 64 bits no menu de kernels. Para conferir o processo e a pasta de saída, pode-se executar uma célula auxiliar:

```
import sys
from pathlib import Path
print(sys.executable)
print(Path.cwd())
```

3. Executa-se a Célula 1 com Shift+Enter e confirma-se a carga do SDK. Executa-se a Célula 2 para definir a função. Com os parâmetros conferidos, executa-se a Célula 3. Run All pode ser utilizado nas execuções seguintes.

Se a Célula 1 exibir erro de carga ou importação, interrompe-se a sequência. Após a correção, reinicia-se o kernel e repetem-se as células na ordem indicada. O notebook captura alguns erros e pode manter definições de uma execução anterior na mesma sessão.

Execução pelo VS Code

Com as extensões Python e Jupyter da Microsoft instaladas, abre-se o notebook existente. Em Select Kernel, seleciona-se PI AF SDK 64 bits quando disponível; em Python Environments, pode-se selecionar diretamente .venv_pi. Executam-se os três blocos na mesma ordem. A troca de kernel exige nova execução das células de definição [6].

Resultados e arquivos

A saída armazenada no notebook original registra o servidor 164.41.16.87 e as contagens abaixo. Trata-se de evidência da execução original; o guia não estabelece uma nova consulta ao servidor.

Tag PI	Registros na execução armazenada
Process.Plant01.Control.FC101.PV	773
Process.Plant01.Control.LC101.PV	94

Os nomes de saída são TAG_Process_Plant01_Control_FC101_PV.csv e TAG_Process_Plant01_Control_LC101_PV.csv. O programa substitui pontos por sublinhados e grava no diretório de trabalho do kernel mostrado por Path.cwd(). Uma nova extração da mesma tag sobrescreve o arquivo de mesmo nome.

Verificação: cada tag apresenta sua contagem, sem mensagens de erro, e o CSV correspondente existe. A mensagem final de sucesso não é suficiente: a função captura exceções por tag e a célula principal pode continuar após uma falha.

12 Validação dos dados adquiridos

Formato e interpretação do CSV PI

O CSV utiliza ponto e vírgula como separador e as colunas Tag, Timestamp e Value. Valores conversíveis em número são formatados com vírgula decimal; estados textuais, como Pt Created, permanecem como texto. Na importação em planilhas, selecionam-se esse separador e a localidade numérica correspondente.

```
Tag;Timestamp;Value
Process.Plant01.Control.FC101.PV;2026-03-25 14:56:00;9,66856384
```

Para análise em pandas, preserva-se Value como texto com keep_default_na=False, inclusive cadeias como NULL [9], e cria-se uma coluna numérica separada. Os estados convertidos em NaN nessa coluna são identificados no tratamento de qualidade, sem substituição automática por zero:

```
import pandas as pd
df = pd.read_csv("TAG_Process_Plant01_Control_FC101_PV.csv",
                 sep=";", dtype={"Tag": str, "Value": str},
                 keep_default_na=False)
df["ValorNumerico"] = pd.to_numeric(
    df["Value"].str.replace(",", ".", regex=False), errors="coerce")
```

Horários e amostragem

O horário OPC é fornecido pelo servidor e não representa o tempo interno de DYNSIM. No notebook PI, Timestamp.LocalTime é formatado em segundos, sem fuso explícito e sem frações de segundo. O CSV também não inclui indicadores de qualidade do AF SDK. Essas propriedades devem ser consideradas ao sincronizar sinais ou avaliar a integridade temporal.

RecordedValues retorna eventos armazenados no arquivo histórico [7]. Os instantes podem ser irregulares e não representam necessariamente todas as amostras originalmente produzidas pela fonte. A obtenção de uma série com passo uniforme requer um procedimento separado de alinhamento ou consulta interpolada.

Paginação e abrangência da evidência

A função usa LOTE_MAXIMO = 50000 na chamada de RecordedValues e, quando o lote atinge esse tamanho, avança o início para o último evento mais 1 ms. Esse valor pertence ao código; não estabelece o valor de ArcMaxCollect do servidor. O notebook preserva os registros extraídos em memória até a escrita do CSV.

O incremento de 1 ms pode excluir eventos no intervalo saltado ou eventos adicionais com o mesmo horário na fronteira. A execução fornecida utilizou apenas um lote por tag e não valida a paginação de períodos extensos. Antes de usar essa rotina em uma extração completa de grande volume, conferem-se os limites do SDK instalado, as fronteiras e as contagens contra uma consulta de referência.

Verificação: os arquivos correspondem às tags e ao período solicitados, estados textuais foram preservados e o referencial temporal foi registrado. A leitura OPC contínua e a paginação PI acima de 50000 registros não foram confirmadas pelas evidências fornecidas.

13 Diagnóstico e retomada da aquisição

Sintoma	Ação de diagnóstico
No suitable Python runtime found	Conferem-se os intérpretes com py -0p e a arquitetura exigida nas seções 2, 8 e 9.
Dispatch Classe não registrada	Confere-se o uso de .venv_opc de 32 bits. Persistindo o erro, verifica-se a instalação e o registro do componente OPC Automation.
OPC encerra após uma consulta	Confere-se a presença de --once. Sem essa opção, registra-se a exceção exibida na console.
PV não acompanha DYNSIM	Conferem-se XRef, Disabled, Model Inputs, Load XRef e o avanço do tempo em RUNNING.
OSIsoft.AFSDK não carrega	Conferem-se a instalação da DLL, caminho_sdk, o ambiente PI selecionado e a compatibilidade do runtime .NET.
ModuleNotFoundError para clr ou pandas	Conferem-se sys.executable e a instalação dos pacotes no ambiente do kernel, conforme a seção 9.
Servidor PI ausente ou incorreto	Confere-se DefaultPIServer ou a seleção explícita de servidor na seção 10.
Tag PI não encontrada ou acesso negado	Conferem-se o nome do ponto, o Data Archive selecionado, a rede ou VPN e as permissões de leitura.
CSV vazio ou arquivo não localizado	Conferem-se período com dados, erros por tag e Path.cwd(). Uma extração vazia pode produzir apenas o cabeçalho.

Retomada das sessões

Para OPC DA, carrega-se DymSim_01.xml em Matrikon, abre-se o caso DYNSIM salvo, carregam-se os motores e o XRef e executa-se Run. Inicia-se então o comando contínuo da seção 8. Os aliases e o OCF são recriados apenas quando a configuração é alterada.

Para PI, confirma-se a rede, abre-se o notebook com o kernel PI e executam-se as células após a revisão de servidor, tags e período. Os CSV anteriores são preservados em outro diretório quando houver necessidade de manter diferentes extrações da mesma tag.

Registro mínimo do experimento

Registram-se o caso ou servidor, os identificadores das variáveis, o intervalo solicitado, o referencial de tempo, as versões das dependências e as contagens obtidas. As versões podem ser arquivadas com pip freeze executado pelo Python de cada ambiente.

14 Fontes e autoria dos materiais

Materiais de origem

A configuração DYNSIM–Matrikon e a leitura pontual OPC foram reconstruídas a partir do guia de conexão e das capturas do caso FC101, registradas em 6 de outubro de 2026. A revisão integrada preserva os nomes de motores, aliases, arquivos e parâmetros confirmados nesse caso.

A rota PI deriva do Tutorial de Leitura de Dados AVEVA PI Server e Python, do notebook Extrator_Dados_PI.ipynb e do README, desenvolvidos por José Matheus Mendes Trindade para o Projeto Refino da Universidade de Brasília. Os arquivos de código fornecidos são mantidos no pacote de apoio; o guia descreve seu comportamento e seus limites sem atribuir nova validação ao servidor.

O nome real do notebook adotado nesta edição é Extrator_Dados_PI.ipynb. Os nomes alternativos presentes no README original não são utilizados nas instruções de execução.

Documentação técnica

[1] Matrikon. MatrikonOPC Server for Simulation User's Manual. Configuração de grupos e aliases.
https://www.matrikonopc.com/downloads/459/index.aspx

[2] Matrikon. MatrikonOPC Explorer User's Manual e Introduction to OPC Tutorial.
https://www.matrikonopc.com/downloads/192/productmanuals/index.aspx
https://www.matrikonopc.com/training/opc-demo-tutorial.pdf

[3] OpenOPC para Python 3.7 e versões posteriores. Dependências e componentes OPC Automation.
https://github.com/j3mg/openopc
https://openopc.sourceforge.net/api.html

[4] Python Software Foundation. Python 3.12.10 e instaladores oficiais.
https://www.python.org/downloads/release/python-31210/

[5] Python.NET. Embedding .NET into Python.
https://pythonnet.github.io/pythonnet/python.html

[6] Microsoft. Jupyter Notebooks in VS Code.
https://code.visualstudio.com/docs/datascience/jupyter-notebooks

[7] AVEVA. PIPoint.RecordedValues Method.
https://docs.aveva.com/bundle/af-sdk/page/html/M_OSIsoft_AF_PI_PIPoint_RecordedValues.htm

[8] IPython. Installing the IPython kernel. Identificador interno e nome visível do kernel.
https://ipython.readthedocs.io/en/stable/install/kernel_install.html

[9] pandas. pandas.read_csv. Preservação de cadeias textuais na importação.
https://pandas.pydata.org/pandas-docs/version/2.2.3/reference/api/pandas.read_csv.html

As instruções adotam dois ambientes Python separados, um para OPC e outro para PI, e especificam o diretório de trabalho como localização efetiva das saídas. A confirmação dos dois diálogos do OPC Configurator identificados na seção 5 permanece necessária para concluir a descrição dos comandos dessa interface.